ÜBERBLICK

Einstieg für Neueinsteiger

Was das Projekt enthält, in welcher Reihenfolge man liest und wo man bei einer Frage weiterliest

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	C8205-Einstieg_fuer_Neueinsteiger.md	0.95-009
Stand	11.11.2026	
Umfang	10 Seiten, A4 hoch	
Erzeugt mit	erzeuge_einstieg_pdf.py	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

C 8205 – Einstieg für Neueinsteiger

Quelle: C8205-Einstieg_fuer_Neueinsteiger.md · im Wortlaut der Quelle

Inhaltsverzeichnis

1. Was ist das Projekt? . . . 3

1.1 Das Regelwerk ist ausführbar . . . 3

1.2 Zwei Wege: Python-Interpreter und Claude-Interpreter . . . 3

2. Die Teilbereiche . . . 4

2.1 Interpreter . . . 4

2.2 Assembler . . . 4

2.3 Bausteininterpretiersystem BIS . . . 4

2.4 Experimenteller Prototyp C8205Z (Zusatztrommeln) . . . 4

3. Drei Ebenen . . . 5

4. Ausprobieren . . . 5

4.1 Als Erstes: das Ratespiel „namen_raten“ laden und spielen . . . 5

4.2 Weitere Möglichkeiten: Test, Assembler und GRAFIK-Start im Überblick . . . 6

5. Begriffe . . . 7

6. Empfohlene Lesereihenfolge . . . 7

7. Wo steht was? . . . 8

8. Technischer Zusammenhang, Befehle und Tests lesen . . . 9

9. Rekonstruktion und historische Sicherheit . . . 9

10. Einstiegspunkte nach Aufgabe . . . 9

11. Kurzfassung . . . 10

C 8205 – Einstieg für Neueinsteiger

Quelle: C8205-Einstieg_fuer_Neueinsteiger.md · im Wortlaut der Quelle

Version: 0.95-009

Zweck: Orientierung für Leser, die das Projekt zum ersten Mal sehen: was es enthält, wie man es startet, in welcher Reihenfolge die Dokumentation zu lesen ist und wo zu einer Frage die Antwort steht. Die technischen Einzelheiten des Rechners stehen in den Fachdokumenten. Wer das Projekt kennt, kann dieses Dokument überspringen. Die README (`README.md`, englisch `README.en.md`) fasst das Projekt auf einer Seite zusammen; dieses Dokument setzt dort an. Wer sofort etwas ausprobieren will, beginnt mit Abschnitt 4.1 (ein fertiges Spiel laden und spielen).

1. Was ist das Projekt?

Das Projekt rekonstruiert und simuliert den Kleinrechner **Cellatron C 8205** (1968, DDR): eine Einadreßmaschine mit **Magnettrommel** als Speicher (128 Spuren × 32 Sektoren, 4096 Worte zu 33 Bit), bedient über Bedienpult, Lochstreifen und Schreibwerk. Zahlen und Adressen werden oktal geschrieben.

Die historische Befehlsverarbeitung wird nicht vereinfacht nachprogrammiert, sondern möglichst genau nachgebildet. Aus den verfügbaren Unterlagen wurde ein **Regelwerk** rekonstruiert: die Knoten des Befehlsablaufs, ihre Ein- und Ausgänge, die Anweisungen an den Knoten, die Wirkung von Registern und Recheneinheiten und die Einbindung der Peripherie. Der Python-Interpreter führt dieses Regelwerk aus.

```
historische Unterlagen
        │
        ▼
rekonstruiertes Regelwerk (.md-Dateien in markdown-dateien/)
        │  Knotengraph und Variablentabelle T2 werden bei jedem Start gelesen
        ▼
Python-Interpreter
        │
        ▼
Testlauf → Testprotokoll
```

1.1 Das Regelwerk ist ausführbar

Zwei Dateien in `markdown-dateien/` sind Dokumentation und zugleich Eingabe des Interpreters:

Datei	Inhalt	Wird gelesen als
`C8205-Interpreter-Eingänge_der_Knoten.md`	Knoten, Typen, Ein- und Ausgänge, Verzweigungen	Knotengraph
`C8205-Interpreter-Beschreibung_der_Knoten.md`	Bedeutung jeder ANWEISUNG-nnn	Variablentabelle T2

Den Ordner bestimmt der Schalter `--mdverz` (Vorgabe: `markdown-dateien/` neben `c8205_interp.py`). Von Hand als Python-Funktion hinterlegt ist nur die **Wirkung** jeder einzelnen ANWEISUNG. Fehlt eine Funktion, meldet der Interpreter einen **Stub**, statt zu raten. Daraus folgt:

- Eine Änderung an diesen beiden Dateien ändert das Verhalten der Simulation, ohne dass Python-Code geändert wird.
- Die Ursache eines unerwarteten Testergebnisses liegt nicht automatisch im Python-Code (siehe Abschnitt 8).

1.2 Zwei Wege: Python-Interpreter und Claude-Interpreter

Dasselbe Testformat lässt sich auf zwei Wegen ausführen. Der **Python-Interpreter** (`c8205_interp.py`) arbeitet den Knotengraphen automatisch und reproduzierbar ab. Beim **Claude-Interpreter** erhält eine KI das komplette Regelwerk und dieselbe Testdatei und arbeitet den Graphen Knoten für Knoten von Hand ab; er dient als unabhängige Gegenprobe. Dieses Dokument behandelt den Python-Interpreter. Zum Claude-Interpreter: `C8205-Interpreter-Projektueberblick.md`, Abschnitt 2, und `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 1b.

2. Die Teilbereiche

2.1 Interpreter

Der Interpreter liegt im Wurzelverzeichnis: `c8205_interp.py` (Einstieg), Beschreibung in `markdown-dateien/C8205-Interpreter-Python_Interpreter.md`. Er ist **nicht** die historische Software des C 8205, sondern das heutige Simulationswerkzeug für die rekonstruierte Hardware.

Zwei Betriebsarten:

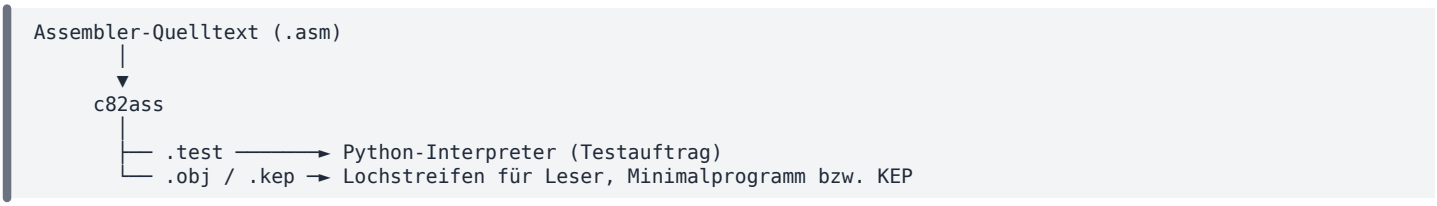
- **TEST**: läuft ohne Fenster und prüft Programme automatisch.
- **GRAFIK** (`c8205_grafik.py`): zeigt Bedienpult und Schreibwerk als Fenster, spielt Töne und bildet die Trommel nach. Beschreibung: `C8205-Interpreter-GRAFIK_Beschreibung.md`.

Stand von GRAFIK: Unter Linux sind die Logiktests und die Hörprüfung durchgeführt. Unter Windows sind die C-Module mit MinGW-w64 gebaut, die Haupttrommel ist getestet und der Mischer (`winmm`) eingeschaltet (Grundgeräusch, Lochbandleser und Stanzer gehört). Schreibwerkton, Dauerbetrieb, Pult-Fenster und andere Python-Versionen sind nicht geprüft. Abnahmestand: `C8205-Interpreter-GRAFIK_Beschreibung.md`, Abschnitt 9.2; Läufe, Log-Schalter und Abnahme: `C8205-Interpreter-GRAFIK_Pruefverfahren.md`.

2.2 Assembler

`Assembler/` enthält den in C geschriebenen Assembler `c82ass`. Er übersetzt Assembler-Quelltext (`.asm`) und erzeugt unter anderem:

- eine `.test`-Datei (Testauftrag für den Python-Interpreter),
- eine `.obj`-Datei (binär codierter Lochstreifen mit Objektcode zur Eingabe mit Kennzeichen, Format des Minimalprogramms),
- mit dem Schalter `--ABSPR` statt der `.obj` eine `.kep`-Datei (Lochstreifen für das Kleine Eingabeprogramm, KEP).



Der Assembler ist ein eigenständiges Teilprojekt. Für den Simulator allein ist seine Dokumentation nicht als Erstes nötig. Einstieg: `Assembler/Dokumentation/C8205_Assembler_Dokumentation.md` (maßgeblich).

2.3 Bausteininterpretiersystem BIS

`Bausteininterpretiersystem BIS/` enthält die Dokumentation des historischen BIS: ein Unterprogrammsystem, das auf dem C 8205 selbst abläuft (Eingabe-, Rechen- und Ausgabebausteine, KEP, Minimalprogramm). Das BIS ist **kein Bestandteil des Python-Interpreters**. Wer zunächst die Funktionsweise des simulierten Rechners verstehen will, kann es überspringen.

2.4 Experimenteller Prototyp C8205Z (Zusatztrommeln)

`C8205Z/` enthält einen **experimentellen Prototyp** der vier Zusatztrommeln des C 8205Z. Er ist kein Teilprojekt: Ohne den Aufrufschalter `--C8205Z` bleibt der Simulator unverändert. Die Anbindung an den Interpreter liegt in `c8205_zt.py` (nur Modus TEST) und in `grafik-interpreter-module/grafik_zt.py` (GRAFIK); die Dateien der Zusatztrommeln liegen in `Trommel/ZT/`. Eckdaten, Zeitmodell und Befehle: `C8205Z/zt/Zusatztrommel_Zeitmodell.md`; die zugehörigen Assemblerprogramme liegen im Projekt ZT-Loader (`Assembler/Projekt_ZT_LOADER/`). Wer zunächst den C 8205 selbst verstehen will, kann den Prototyp überspringen.

3. Drei Ebenen

Die Dokumentation behandelt drei Ebenen, die nicht verwechselt werden sollen:

Ebene	Gegenstand
A: der historische Rechner	Rechenwerk, Leitwerk, Register, Magnettrommel, Bedienpult, Lochstreifen, Schreibwerk, Befehle
B: die Rekonstruktion	wie aus den Unterlagen ein ausführbares Regelwerk entstand: Knotengraph mit Variablentabelle T2
C: die heutige Simulation	Python, Dateien, Testskripte, virtuelle Geräte

4. Ausprobieren

Voraussetzung: Python 3 (geprüft mit 3.11 bis 3.13), keine zusätzlichen Pakete. Ein C-Compiler (`gcc`, `make`) wird nur zum Neubau des Assemblers gebraucht; das fertige Binary liegt in `Assembler/_Linux/c82ass`. Unter Linux vor jedem Aufruf `export LC_ALL=C.UTF-8` setzen und das Zip nicht mit `unzip` entpacken, sondern mit Python-`zipfile` (`unzip` macht in der POSIX-Locale aus Umlauten `#U00e4`, und der Interpreter bricht ab; der Einzeiler, der auch die Rechte der Skripte setzt, steht in `CLAUDE.md`, Abschnitt „Schnellstart“).

4.1 Als Erstes: das Ratespiel „namen_raten“ laden und spielen

`namen_raten` ist ein fertiges Programm für den C 8205: Der Rechner wählt aus fünf Namen mit je sieben Buchstaben (Michael, Barbara, Susanne, Andreas, Claudia) einen aus und nennt den 1. Buchstaben; man tippt die übrigen Buchstaben am Schreibwerk (SW). Das Programm wird so geladen, wie es am Gerät geschieht: Das Kleine Eingabeprogramm (KEP, Spur 0) liest den binär codierten Lochstreifen `namen_raten.kep` vom Leser 1 und schreibt das Programm auf die Trommel. Alle Befehle unten gelten im Projektstamm (dort, wo `c8205_interp.py` liegt). Für das Pult-Fenster wird das **Gesamt-Zip** gebraucht: die Bilder des Pult-Fensters (`pult/`) fehlen in der Claude-Variante.

Schritt 1: das KEP auf die Trommel bringen. Der TEST-Interpreter schreibt nur das KEP auf die Trommel und führt nichts aus (`--LOADONLY`):

```
export LC_ALL=C.UTF-8      # nur Linux/macOS
python3 c8205_interp.py --MODUS=TEST Tests/bsp_5_18_kep.test --LOADONLY
```

Der Lauf leert zuerst die Trommel `Trommel/trommel.txt`; danach steht nur das KEP auf ihr. Er endet mit Rückgabecode 0.

Schritt 2: den Lochstreifen in den Ordner von Leser 1 legen. Dort darf nur diese eine Datei liegen (bei mehreren nimmt der Leser die nach Änderungszeit nächste):

```
cp "Lochstreifen/im_KEP-Format/Spiele/namen_raten.kep" virtuelle_Geraete/Leser1/          # Linux/macOS
copy "Lochstreifen\im_KEP-Format\Spiele\namen_raten.kep" virtuelle_Geraete\Leser1\        # Windows
```

Alternativ lässt sich derselbe Streifen aus dem Quelltext erzeugen (das Ergebnis ist byte-gleich mit der Datei im Ordner `Spiele/`):

```
cd Assembler
./c82ass.sh -p Projekt_1/namen_raten -s namen_raten.asm -o ../virtuelle_Geraete/Leser1 --ABSPR
cd ..
```

Die Meldung zur Bahn 0000 bis 0037 ist hier Absicht: Zelle 0000 trägt den Sprung zur Startadresse 2000 (oktal).

Schritt 3: GRAFIK starten. Es öffnen sich das Pult-Fenster, das SW-Fenster (Schreibwerk) und, durch `--HWCHECK`, ein Fenster mit BZ, BR, Spur/Sektor, AC und AR:

```
./c8205_grafik.sh --HWCHECK        # Windows: c8205_grafik.bat --HWCHECK
```

Ohne Töne (und ohne die Wartezeit des Hochlaufs): zusätzlich `--STUMM`. Das Lesetempo des Lochstreifenlesers bleibt auch mit `--STUMM` erhalten (Schritt 4, Punkt 3).

Schritt 4: den Rechner einschalten und das Programm einlesen (bedient wird mit der Maus am Pult-Fenster):

1. Den Schlüsselschalter unter der Lampe N anklicken. Der Schalter wird grün, die Lampe B rot; nach dem Hochlauf (rund 11 Sekunden, mit `--STUMM` sofort) wird B grün, und der Rechner ist eingeschaltet.

2. Die Tasten in dieser Reihenfolge: `H`, `G`, `M` halten (M+: Umschalt+Klick auf M, oder Umschalt+M auf der Tastatur), `E`, `E`, `M` lösen (M−: nochmals Umschalt+Klick), die Kanaltaste `K1`, `R`.
3. Das KEP liest jetzt den Streifen aus `virtuelle_Geraete/Leser1/` und schreibt die Worte auf die Trommel; am Streifenende bleibt es stehen. Der gelesene Streifen wandert nach `virtuelle_Geraete/Leser1/LOCHSTREIFENKORB/`. **Das Einlesen dauert rund 1:40 Minuten** (gemessen). Der Streifen hat 12 139 Zeichen, der Standardleser 303 liest 130 Zeichen/s (rechnerisch 93 Sekunden), und dieses Tempo gilt auch mit `--STUMM`. Wer nicht warten will, startet GRAFIK in Schritt 3 zusätzlich mit `--LS-LESER-TYP FS751` (optoelektronischer Leser, 750 Zeichen/s; rechnerisch rund 16 Sekunden).

Schritt 5: das Spiel starten und spielen.

1. `H` drücken (die Wippe T-SW ist nur bei H bedienbar).
2. Die Wippe T-SW von `T` auf `SW` stellen (Taste SW am Pult, auf der Tastatur `S`): Die Buchstaben kommen aus dem SW-Fenster.
3. `G` drücken (stellt das Befehlsregister BR auf „0000 000“), dann `R` (führt es aus): Der Ablauf springt über den Sprung in Zelle 0000 zur Startadresse 2000 (oktal), und das Programm läuft an.
4. Im SW-Fenster erscheint zum Beispiel:

```
Rate mal, welchen Namen ich mir gemerkt habe.
Mit der Eingabe eines Punktes beendest du das Programm.
1. Buchstabe ist:  M
   Falsch, noch ein Versuch.
   Du musst deine Fantasie viel mehr schulen.
...
```

Die Buchstaben tippt man im SW-Fenster (Groß- und Kleinschreibung gelten gleich). Jeder Buchstabe darf einmal wiederholt werden; ist auch der zweite Versuch falsch, kommt „Du musst deine Fantasie viel mehr schulen.“, dann drei Leerzeilen und ein neues Spiel. Sind alle Buchstaben erraten, kommt „Toll, du bist fantasiebegabt.“, wieder drei Leerzeilen und ein neues Spiel. Ein Punkt beendet das Programm zu jeder Zeit. Umlaute und ß gibt es am SW nicht.

Ende und Wiederholung. `H` unterbricht das Programm jederzeit. Ein Klick auf den Schlüsselschalter schaltet den Rechner aus; nach dem Ausklang (rund 2 Sekunden) endet GRAFIK mit Rückgabecode 0. Das Spiel bleibt auf der Trommel stehen (die Trommel ist persistent): Zum erneuten Spielen genügt Schritt 3, Schritt 4 Punkt 1 und Schritt 5; das Einlesen von rund 1:40 Minuten entfällt. Wer das Programm neu einlesen will, legt den Streifen aus dem `LOCHSTREIFENKORB` zurück nach `virtuelle_Geraete/Leser1/`. Testläufe können die Trommel leeren oder überschreiben (auch `testscripte/compute_all_tests.sh`); dann sind die Schritte 1 bis 4 zu wiederholen.

4.2 Weitere Möglichkeiten: Test, Assembler und GRAFIK-Start im Überblick

Test ausführen (aus dem Projektstamm):

```
python3 -I c8205_interp.py --MODUS=TEST Tests/trommel-001_leer.txt
```

Mit Wrapper (liegt im Projektstamm, in beiden Zip-Fassungen; Testdatei an beliebiger Stelle der Argumente, Protokoll nach `Test-Protokolle/`): `./c8205_interpreter.sh Tests/trommel-001_leer.txt` (Linux/macOS) bzw. `c8205_interpreter.bat Tests\trommel-001_leer.txt` (Windows).

Erwartet: Ausgabe endet mit `Test wahr`. Rückgabecodes: 0 Test wahr, 4 ohne Prüfung, 6 Test falsch, 8 Laufzeitfehler, 16 formale Prüfung negativ, 2 Aufruf falsch. Das Protokoll schreibt der direkte Aufruf nach `Test-Protokolle/<Testname>/protokoll.txt`, der Wrapper nach `Test-Protokolle/<Testname>.log`; die letzte Ausgabezeile „Protokolldatei: …“ nennt den Pfad. Die Trommeldatei `Trommel/trommel.txt` wird bei jedem Lauf verändert; Tests setzen dafür `#TROMMEL=LEER`.

Alle Tests (etwa 15 bis 20 Sekunden): `testscripte/compute_all_tests.sh`. Die Ausgabe ist eine Zeile „compute_all_tests: N Laeufe, M Abweichungen“; Rückgabecode 0 bedeutet keine Abweichung.

Assemblieren (aus dem Ordner `Assembler/`):

```
./c82ass.sh -p Claude/mini -s mini --WARN=ALLE --no-color
```

Rückgabecode 0 bedeutet fehlerfrei. Erzeugt werden `.obj .mqo .sym .dmp .log .test` im Projektordner. Das Skript `Assembler/asm_und_test.sh` assembliert und führt den Test in einem Schritt aus (`Assembler/asm_und_test.sh Claude/mini mini`); die Sollwerte des Tests stehen im `.asm` zwischen `;#PRUEFE-START` und `;#PRUEFE-ENDE` (Vorlage `Assembler/Claude/mini/mini.asm`).

GRAFIK (aus dem Projektstamm):

```
python3 c8205_grafik.py --STUMM        # Pult- und Schreibwerk-Fenster, ohne Töne
python3 c8205_grafik.py                # mit Tönen (Linux: paplay, pw-cat, aplay oder ffplay nötig)
./c8205_grafik.sh --STUMM              # Wrapper, aus jedem Ordner; Windows: c8205_grafik.bat
```

Alle Schalter und die Geräteordner beschreibt `C8205-Interpreter-GRAFIK_Beschreibung.md` (Abschnitte 1, 2 und 6).

5. Begriffe

Begriff	Bedeutung
AC, BZ, AR	Akkumulator (33 Bit), Befehlszähler (12 Bit), Adreßregister (12 Bit); weitere Register und Flags (V1, V2, US, BS, FE …): `C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`
Knoten, Knotengraph	Schritt des Befehlsablaufs und der Graph aller Schritte (rund 80 Knoten); ANWEISUNG-nnn ist eine Operation an einem Knoten
Variablentabelle T2	Zuordnung der ANWEISUNGen zu ihrer Bedeutung
Stub	vom Interpreter gemeldete, noch nicht gesicherte Stelle des Regelwerks
Testauftrag	Eingabe eines Testlaufs: Trommeldatei (Speicherinhalt, oktal), Tastenfolge als Ausgangszustand (z. B. `H G R`), Testmarken für Diagnoseausgaben, optional Prüfungen mit Sollwerten
KEP	Kleines Eingabeprogramm auf Spur 0 (Adressen 0000 bis 0037); verarbeitet nur die Zeichen 0, 1, Punkt, Doppelpunkt und Zwischenraum
Minimalprogramm	Grundbaustein des BIS; liest Befehle, Adressen und Konstanten über die Eingabekennzeichen K1 und K2
K1K2	Kennzeichen am Anfang einer Eingabezeile; bestimmen die Aufbereitung (z. B. 00 absolut, 01 bezogen auf die 1. Leitadresse, 20 Relativadresse setzen)
BIS	Bausteininterpretiersystem (Abschnitt 2.3)

6. Empfohlene Lesereihenfolge

Nicht alphabetisch lesen. Alle Dateien außer diesem Dokument liegen in `markdown-dateien/`.

Schritt	Lesen	Ergebnis
1	dieses Dokument	Überblick über die Teile des Projekts
2	`C8205-Interpreter-Projektueberblick.md`	Gesamtidee, Verweise auf die Fachdokumente
3	`C8205-Interpreter-Überblick_und_Gesamtaufbau.md`	technischer Aufbau des C 8205
4	`C8205-Projektstruktur.md`	welche Datei wo liegt (Landkarte, kein Lehrbuch)
5	`C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`	Register und Datenstrukturen
6	`C8205-Interpreter-Befehlsliste.md`	Befehle des Rechners
7	`C8205-Interpreter-Eingänge_der_Knoten.md` und `C8205-Interpreter-Beschreibung_der_Knoten.md`	Kern der Rekonstruktion: vom Maschinenbefehl zum Ablauf
8	`C8205-Interpreter-Python_Interpreter.md`	Ausführung des Regelwerks in Python
9	`C8205-Interpreter-Durchführung_eines_Tests.md`	Testläufe nachvollziehen

Beim ersten Durchgang entbehrlich, als Nachschlagewerk später nötig: `C8205-Interpreter-PDF.md`, `C8205-Versionierung.md` (außer vor Änderungen an Dateien), `C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`, `C8205-Interpreter-Kodetabellen.md`, `C8205-Interpreter-Magnettrommel.md`, `C8205-Interpreter-Lochstreifenleser.md`, `baz-wirkungstabelle.md` (Zwischenspeicher des Claude-Interpreters), `C8205-Interpreter-Bedienpult_mit_Schaltern.md`, `C8205-Interpreter-GRAFIK_Beschreibung.md` und `C8205-Interpreter-GRAFIK_Pruefverfahren.md` (nur für GRAFIK), das Programmierhandbuch mit seinen Anhängen A und B und `C8205-Schleifenoptimierung_Eingabeschleife.md` (nur zum Schreiben und Optimieren von Assemblerprogrammen), die Dokumente zum Prototyp C8205Z unter `C8205Z/zt/` (Abschnitt 2.4), die Assembler-Dokumentation und die BIS-Dokumentation. Verweise wie „siehe Datenstruktur und

Verarbeitungseinheiten“ führen auf die nächste Ebene; die Dokumentation ist ein Nachschlagewerk mit Querverweisen, kein einzelnes Lehrbuch.

7. Wo steht was?

Die vollständige Dokumentenlandkarte steht in `C8205-Interpreter-Projektueberblick.md`, Abschnitt 6. Alle Dateien liegen in `markdown-dateien/`, sofern kein anderer Pfad angegeben ist.

Frage	Dokument
Gesamtprojekt	`C8205-Interpreter-Projektueberblick.md`
Aufbau, Ordner (`Tests/`, `Test-Protokolle/`, `Trommel/` …), Skripte	`C8205-Projektstruktur.md`
Wie funktioniert der Rechner grundsätzlich?	`C8205-Interpreter-Überblick_und_Gesamtaufbau.md`
Register	`C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md`
Befehle	`C8205-Interpreter-Befehlsliste.md`
Aufbau der Knoten / Eingänge der Knoten	`C8205-Interpreter-Beschreibung_der_Knoten.md` / `C8205-Interpreter-Eingänge_der_Knoten.md`
Python-Interpreter	`C8205-Interpreter-Python_Interpreter.md`
Claude-Interpreter	`C8205-Interpreter-Projektueberblick.md`, Abschnitt 2
Test ausführen oder schreiben	`C8205-Interpreter-Durchführung_eines_Tests.md`
Magnettrommel, Lochstreifenleser, Zeichenkodes	`C8205-Interpreter-Magnettrommel.md`, `C8205-Interpreter-Lochstreifenleser.md`, `C8205-Interpreter-Kodetabellen.md`
Lochstreifen-Hilfsprogramme	`C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`
Bedienpult	`C8205-Interpreter-Bedienpult_mit_Schaltern.md`
GRAFIK-Modus	`C8205-Interpreter-GRAFIK_Beschreibung.md`
GRAFIK-Prüfverfahren (Läufe, Log-Schalter, Abnahme)	`C8205-Interpreter-GRAFIK_Pruefverfahren.md`
Laufzeit, Trommelzeit, Lage einer Schleife auf der Trommel	`C8205-Programmierhandbuch-Anhang_A_Trommelzeit_GRAFIK.md`, `C8205-Schleifenoptimierung_Eingabeschleife.md`
Zusatztrommeln (Prototyp C8205Z)	`C8205Z/zt/Zusatztrommel_Zeitmodell.md` (Abschnitt 2.4)
Versionsnummern der Dateien	`C8205-Versionierung.md`
Assembler	`Assembler/Dokumentation/C8205_Assembler_Dokumentation.md`
Standardmakros des Assemblers	`Assembler/Dokumentation/C8205_Standardmakros_c82ass.md`
Assemblerprogramme schreiben (auch mit KI)	`C8205-Programmierhandbuch_fuer_Claude.md` (Wurzelverzeichnis), Karte `CLAUDE.md`
BIS, Objektcode	`Bausteininterpretiersystem BIS/markdown-dateien/C8205-Interpreter-BIS.md` und `C8205-Interpreter-Objektdatei_Listing.md`
Anschluss eines Assemblerprogramms an das BIS-Minimalprogramm	`C8205-Programmierhandbuch-Anhang_B_BIS_Anschluss.md`
Stand der laufenden Arbeit	Datei `aufgabe` im Projektstamm (eine leere Datei heißt: kein Altbestand)

Weitere Ordner: `Hilfsprogramme/` (Lochstreifen-Hilfsprogramme), `testscripte/` (Testskripte, Prüfungen), `Lochstreifen/` (fertige Lochstreifen- und Ladedateien, nach Format geordnet: `im_KEP-Format/`, `im_BIS-Format/`, `das_KEP_im_TEST-Format/`, `Datenformat/`), `Testausfuehrungen/` (Skripte `testwahl.sh`, `kep_test.sh`, `interpreter_test.sh`, `bis_test.sh`, dazu die Testtabelle `test_tabelle*`, die nur im Gesamt-Zip liegt), `c-module/` und `grafik-interpreter-module/` (Bausteine des GRAFIK-Interpreters; in `c-module/` liegt das gebaute C-Modul der Trommel, ohne das GRAFIK nicht startet), `C8205Z/` (Abschnitt 2.4), `Benutzer-Scripte/` (Skripte des Projektinhabers für den eigenen Rechner mit festen Pfaden; nicht Teil der Testläufe) und `Regressionstest.zip` (Regressionstest des Assemblers, nur nach einer Änderung am Assembler-Quelltext nötig). Das Gesamt-Zip enthält zusätzlich alle PDFs samt `pdf-layout-scripte/`, `pult/` (Bilder des Pult-Fensters), `ton/`, `README.en.md` und `Test-Protokolle.zip`; in der

schlanken Claude-Variante (Name endet auf „Claude“) fehlen sie, alle Tests laufen mit ihr gleich. Einzelheiten: Handbuch für Claude, Abschnitt 11.

8. Technischer Zusammenhang, Befehle und Tests lesen

```
C-8205-Programm
      │
      ▼
   Testauftrag (Trommelinhalt, Tastenfolge, …)
      │
      ▼
 Python-Interpreter
      │
      ▼
 Befehlsverarbeitung → Knotengraph (Knoten, Eingänge, Anweisungen, Übergänge)
      │
      ▼
 Register / Speicher / Geräte → Ergebnis
```

Der Knotengraph ist der zentrale Punkt. Der Interpreter führt nicht abstrakt `ADD → AC = AC + Speicher` aus, sondern vollzieht die rekonstruierte Befehlsverarbeitung nach dem Regelwerk nach. Alle Zeiten der echten Hardware (Wortzeit, Wartezeit der Trommel) sind in der Simulation 0; die Reihenfolge der Ereignisse bleibt erhalten. Läufe lassen sich dadurch als Regressionstests wiederholen.

Einen einzelnen Befehl nachvollziehen: nicht im Python-Quelltext beginnen, sondern in dieser Reihenfolge: Befehlsliste → Beschreibung des Befehls → relevante Knoten → Knoteneingänge → Register und Datenstrukturen → Python-Implementierung der Anweisungen → Test.

Fehlersuche bei einem Test: Reihenfolge der Fragen: Ausgangszustand, Trommelzustand und Registerwerte richtig? Richtiger Befehl geladen? Welcher Knoten wurde zuletzt ausgeführt, welche Anweisung war dort problematisch? War die Regelwerksbeschreibung eindeutig? Erst danach: Fehler im Python-Interpreter. So wird ein Problem nicht vorschnell dem Python-Code zugeschrieben.

Die Teile eines Tests liegen in diesen Ordnern:

Ordner	Inhalt
`Tests/`	Testaufträge (`.test` und `.txt`; der Assembler erzeugt `.test`)
`Test-Ergebnisvergleiche/`	Soll-Dateien (Namen meist auf `-ausgabe.txt` oder `_erwartet.txt`), mit denen Tests ihre Ausgabe vergleichen
`Test-Protokolle/`	Protokolle der Interpreterläufe; der Ordner entsteht erst beim Lauf; im Gesamt-Zip liegt er als `Test-Protokolle.zip`, in der Claude-Variante fehlt er
`Test-virtuelleGeraete/`	virtuelle Geräte des TEST-Interpreters: `Leser1/`, `Leser1_02/`, `Leser2/`, `Stanzer1/`, `SW-Ausgabe/`, `T-SW-Eingabe/`
`virtuelle_Geraete/`	virtuelle Geräte des GRAFIK-Interpreters: `Leser1/`, `Leser2/`, `Stanzer1/`
`Lochstreifen/`	fertige Lochstreifen- und Ladedateien, nach Format geordnet
`Trommel/`	die gemeinsame Trommel (`trommel.txt`) für TEST und GRAFIK

9. Rekonstruktion und historische Sicherheit

Zu unterscheiden sind: historisch belegt, rekonstruiert, angenommen und technische Implementierung. Ist eine Stelle der Unterlagen nicht eindeutig, darf die Implementierung diese Unsicherheit nicht unbemerkt in einen scheinbar sicheren Fakt verwandeln. Dafür meldet der Interpreter unklare Stellen als offenen Punkt („Stub“), statt sie stillschweigend zu „reparieren“. Für Änderungen und Erweiterungen ist diese Unterscheidung besonders wichtig.

10. Einstiegspunkte nach Aufgabe

Aufgabe	Beginnen mit
ein fertiges Programm ausprobieren	Abschnitt 4.1 (Spiel `namen_raten`, alle Befehle stehen dort)

Aufgabe	Beginnen mit
einen Test ausführen	`C8205-Interpreter-Durchführung_eines_Tests.md`; Assembler- oder BIS-Kenntnisse sind dafür nicht nötig
einen Maschinenbefehl verstehen	Vorgehen aus Abschnitt 8
den Python-Interpreter ändern	`C8205-Interpreter-Python_Interpreter.md`, danach die Dokumentation des betroffenen Regelwerks. Das Verhalten des Knotengraphen steht in den beiden Knoten-Dateien (Abschnitt 1.1); in Python liegt nur die Wirkung der einzelnen Anweisungen.
einen neuen Test schreiben	`C8205-Interpreter-Durchführung_eines_Tests.md`, danach vorhandene Dateien in `Tests/`
Assemblerprogramme schreiben	`Assembler/Dokumentation/`, `C8205-Programmierhandbuch_fuer_Claude.md` und die vorhandenen `.asm`-Beispiele; Aufruf: `Assembler/asm_und_test.sh` (Abschnitt 4)
den Assembler (C-Quelltext) ändern	`Assembler/Sourcecode/`, danach der Regressionstest (Programmierhandbuch, Abschnitt 8, Schritt 8). Maßgeblich ist die Assemblerdokumentation (`.md`); bei jeder Änderung am Assembler wird sie mitgeliefert.
GRAFIK-Fenster, Pult oder Ton ändern oder prüfen	`C8205-Interpreter-GRAFIK_Beschreibung.md` und `C8205-Interpreter-GRAFIK_Pruefverfahren.md`; dafür wird das Gesamt-Zip gebraucht (`pult/` und `ton/` fehlen in der Claude-Variante)
die Laufzeit eines Programms messen	`C8205-Programmierhandbuch-Anhang_A_Trommelzeit_GRAFIK.md`
das BIS verstehen	`Bausteininterpretiersystem BIS/`, nicht die Python-Interpreter-Dokumentation
eine `.md`-Datei ändern	`C8205-Versionierung.md` lesen. Jede Änderung erhöht den Build (`zzz` in `y.xx-zzz`) um 1; Haupt- und Unterversion erhöht nur der Projektinhaber auf ausdrückliche Aufforderung.

11. Kurzfassung

```
HISTORISCHER C 8205 → REKONSTRUKTION → REGELWERK → KNOTENGRAPH → PYTHON-INTERPRETER → TESTS
```

Wichtig sind fünf Aussagen:

1. Der C 8205 ist der historische Rechner.
2. Das Regelwerk (die `.md`-Dateien) beschreibt die rekonstruierte Befehlsverarbeitung.
3. Der Knotengraph ist der zentrale Ablauf dieser Befehlsverarbeitung und steht als Datei im Regelwerk.
4. Der Python-Interpreter liest diesen Knotengraphen und führt ihn aus.
5. Tests zeigen, ob die Rekonstruktion und ihre Implementierung wie erwartet funktionieren.

Lesereihenfolge: Projektüberblick → Gesamtaufbau → Projektstruktur → Datenstrukturen → Befehle → Knoten → Interpreter → Tests. Assembler, BIS und der Prototyp C8205Z sind eigene Wege und müssen für das Verständnis des Interpreters nicht gelesen werden.